

QSM368ZP-WF&SG368Z 系列

Linux&Ubuntu&OpenWrt

编译&烧录指导

智能产品

版本：1.2

日期：2025-11-26

状态：受控文件



上海移远通信技术股份有限公司（以下简称“移远通信”）始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司
上海市闵行区田林路 1016 号科技绿洲 3 期（B 区）5 号楼 邮编：200233
电话：+86 21 5108 6236 邮箱：info@quectel.com

或联系我司当地办事处，详情请登录：<https://www.quectel.com.cn/contact>。

如需技术支持或反馈我司技术文档中的问题，请随时登录网址：
<https://www.quectel.com.cn/contact?tab=t> 或发送邮件至：support@quectel.com。

前言

移远通信提供该文档内容以支持客户的产品设计。客户须按照文档中提供的规范、参数来设计产品。同时，您理解并同意，移远通信提供的参考设计仅作为示例。您同意在设计您目标产品时使用您独立的分析、评估和判断。在使用本文档所指导的任何硬软件或服务之前，请仔细阅读本声明。您在此承认并同意，尽管移远通信采取了商业范围内的合理努力来提供尽可能好的体验，但本文档和其所涉及服务是在“可用”基础上提供给您的。移远通信可在未事先通知的情况下，自行决定随时增加、修改或重述本文档。

使用和披露限制

许可协议

除非移远通信特别授权，否则我司所提供硬软件、材料和文档的接收方须对接收的内容保密，不得将其用于除本项目的实施与开展以外的任何其他目的。

版权声明

移远通信产品和本协议项下的第三方产品可能包含受移远通信或第三方材料、硬软件和文档版权保护的相关资料。除非事先得到书面同意，否则您不得获取、使用、向第三方披露我司所提供的文档和信息，或对此类受版权保护的资料进行复制、转载、抄袭、出版、展示、翻译、分发、合并、修改，或创造其衍生作品。移远通信或第三方对受版权保护的资料拥有专有权，不授予或转让任何专利、版权、商标或服务商标权的许可。为避免歧义，除了正常的非独家、免版税的产品使用许可，任何形式的购买都不可被视为授予许可。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法侵权行为，移远通信有权追究法律责任。

商标

除另行规定，本文档中的任何内容均不授予在广告、宣传或其他方面使用移远通信或第三方的任何商标、商号及名称，或其缩略语，或其仿冒品的权利。

第三方权利

您理解本文档可能涉及一个或多个属于第三方的硬软件和文档（“第三方材料”）。您对此类第三方材料的使用应受本文档的所有限制和义务约束。

移远通信针对第三方材料不做任何明示或暗示的保证或陈述，包括但不限于任何暗示或法定的适销性或特定用途的适用性、平静受益权、系统集成、信息准确性以及与许可技术或被许可人使用许可技术相关的不侵犯任何第三方知识产权的保证。本协议中的任何内容都不构成移远通信对任何移远通信产品或任何其他硬件、设备、工具、信息或产品的开发、增强、修改、分销、营销、销售、提供销售或以其他方式维持生产的陈述或保证。此外，移远通信免除因交易过程、使用或贸易而产生的任何和所有保证。

隐私声明

为实现移远通信产品功能，特定设备数据将会上传至移远通信或第三方服务器（包括运营商、芯片供应商或您指定的服务器）。移远通信严格遵守相关法律法规，仅为实现产品功能之目的或在适用法律允许的情况下保留、使用、披露或以其他方式处理相关数据。当您与第三方进行数据交互前，请自行了解其隐私保护和数据安全政策。

免责声明

- 1) 移远通信不承担任何因未能遵守有关操作或设计规范而造成损害的责任。
- 2) 移远通信不承担因本文档中的任何因不准确、遗漏、或使用本文档中的信息而产生的任何责任。
- 3) 移远通信尽力确保开发中功能的完整性、准确性、及时性，但不排除上述功能错误或遗漏的可能。除非另有协议规定，否则移远通信对开发中功能的使用不做任何暗示或法定的保证。在适用法律允许的最大范围内，移远通信不对任何因使用开发中功能而遭受的损害承担责任，无论此类损害是否可以预见。
- 4) 移远通信对第三方网站及第三方资源的信息、内容、广告、商业报价、产品、服务和材料的可访问性、安全性、准确性、可用性、合法性和完整性不承担任何法律责任。

版权所有 © 上海移远通信技术股份有限公司 2025，保留一切权利。

Copyright © Qectel Wireless Solutions Co., Ltd. 2025.

文档历史

修订记录

版本	日期	作者	变更描述
-	2024-07-12	Szymon Yu	文档创建
1.0	2024-07-18	Szymon Yu	受控版本
1.1	2025-02-26	Ivan Zujovic	- 新增 Ubuntu 编译环境版本及备注（第 2.1 章）。 - 新增 OpenWrt 操作系统的编译及烧录相关说明。 - 新增关于 FTP服务器的用途备注（第 2.2.1 章）。 - 新增 OpenWrt 操作系统下的开发示例（第 4.3 章）。
1.2	2025-11-26	Damian Li	- 代码块增加换行符提示。 - 新增关于项目开发注意事项和PDF 复制代码可能导致非预期执行的说明备注（第 1 章）。 - 新增 Kernel 6.1 编译命令相关内容（第 3.2 章）。

目录

文档历史	3
目录	4
表格索引	5
图片索引	6
1 引言	7
2 开发环境	8
2.1. 编译环境.....	8
2.2. 安装和启动工具.....	8
2.2.1. 在 Windows 上安装工具.....	8
2.2.2. 在 Ubuntu 上安装工具	9
2.2.3. 在 Ubuntu 上使用 Docker	9
2.2.3.1. 安装 Docker	9
2.2.3.2. 将 Ubuntu 18.04/20.04/22.04 镜像导入 Docker.....	10
3 编译与烧录	14
3.1. 访问 GitLab 克隆代码.....	14
3.2. 编译命令.....	14
3.2.1. Linux 系统.....	14
3.2.1.1. Kernel 5.10 及之前	14
3.2.1.2. Kernel 6.1	16
3.2.2. Ubuntu 系统.....	17
3.2.3. OpenWrt 系统.....	18
3.3. 构建、加载及烧录固件包.....	19
4 开发示例	23
4.1. Linux 系统下的开发示例	23
4.2. Ubuntu 系统下的开发示例	24
4.3. OpenWrt 系统下的开发示例	26

5 附录 参考文档及术语缩写.....28

表格索引

表 1: 编译环境 8

表 2: 参考文档28

表 3: 术语缩写28

图片索引

图 1: 导入到 Docker 的镜像 (Linux 系统)	10
图 2: 导入到 Docker 的镜像 (Ubuntu 系统)	11
图 3: 导入到 Docker 的镜像 (OpenWrt 系统)	11
图 4: Docker 容器 ID (Linux 系统)	12
图 5: Docker 容器 ID (Ubuntu 系统)	12
图 6: Docker 容器 ID (OpenWrt 系统)	12
图 7: 镜像文件	20
图 8: BOOT 开关	20
图 9: 固件选择	21
图 10: 烧录结果	21
图 11: 打开编译宏	24
图 12: 示例	25

1 引言

本文档主要用于指导客户如何在移远通信 QSM368ZP-WF 产品和 SG368Z 系列模块上获取、编译和加载智能产品的软件，包括：

- 建立开发环境并安装相关工具；
- 编译 Linux、Ubuntu 或 OpenWrt 代码，并将生成的固件包烧录至设备中。

备注

1. 本文档适用于运行Linux操作系统的QSM368ZP-WF产品和运行 Linux 或 Ubuntu 或 OpenWrt 操作系统的 SG368Z 系列模块。
2. SG368Z 系列模块本身无法预装 OpenWrt 系统，仅提供 SDK 及开发指导供客户二次开发。若有问题，请联系移远通信技术支持。
3. <https://github.com/quectel-smart/QSM368ZP-WF>用于存放工具、驱动和镜像等相关资料供用户下载使用，无其他用途。
4. 由于 PDF 格式特性，文档中部分代码块可能因页面宽度限制自动换行。直接从 PDF 复制代码到编辑器时，可能引入非预期的换行符（\n），导致代码无法正常运行。
建议操作：
若需从 PDF 复制，粘贴后请注意完成如下动作：
 - 手动删除复制粘贴后代码中的多余换行符。
 - 或通过代码编辑器的替换功能批量删除多余换行符（注意保留语义换行）。

2 开发环境

2.1. 编译环境

表 1: 编译环境

描述	版本	作用
Linux 开发工作站的性能高于在一般桌面系统上运行 Ubuntu 64 位操作系统的最低要求。为加快编译时间，建议使用性能强大的 PC，该 PC 应当符合如下规格： CPU: Intel i7-2600 @ 3.4 GHz 或以上 内存: 16 GB RAM 或以上 硬盘: 200 GB 或以上（首选 SSD 加速）	Ubuntu 64 位 18.04 LTS/ Ubuntu 64 位 20.04 LTS/ Ubuntu 64 位 22.04 LTS	Linux 编译机
Windows 系统	Windows 10 或更高版本	烧录固件包

备注

1. 推荐使用 Ubuntu 64 位 18.04 LTS（用于编译 Linux 系统）或 64 位 20.04 LTS（用于编译 Ubuntu 系统）或 64 位 22.04 LTS（用于编译 OpenWrt 系统）。移远通信已经验证了 Ubuntu 64 位 18.04 LTS、Ubuntu 64 位 20.04 LTS 和 Ubuntu 64 位 22.04 LTS 版本的编译工作，对于其他 Ubuntu 版本，移远通信暂未验证。
2. 编译前需要获取相关代码，请联系移远通信技术支持获取 GitLab 访问权限来下载 SDK。

2.2. 安装和启动工具

2.2.1. 在 Windows 上安装工具

1. RKDevTool
2. USB 驱动

<https://developer.quectel.com/doc/files/QSM368ZP/tool.zip> 本次比赛相关资料在该路径下下载。

2.2.2. 在 Ubuntu 上安装工具

若使用的 Ubuntu 版本满足第 2.1 章的编译环境要求，可以执行如下命令在 Ubuntu 上安装相关工具，然后参考第 3 章进行编译和烧录步骤。

```
sudo apt-get install repo git ssh make gcc libssl-dev liblz4-tool expect g++ \
patchelf chrpath gawk texinfo chrpath diffstat binfmt-support qemu-user-static \
live-build bison flex fakeroot cmake gcc-multilib g++-multilib unzip \
device-tree-compiler python-pip libncurses5-dev zstd rsync device-tree-compiler bc time
```

备注

若编译主机默认的系统 shell 是 Dash 而非 Bash，请按照如下步骤将符号链接/bin/sh→dash 更改为/bin/sh→bash：

1. 执行 `sudo rm /bin/sh` 命令删除符号链接/bin/sh；
2. 执行 `sudo ln -s /bin/bash /bin/sh` 命令，创建一个指向/bin/bash 的符号链接，并命名为/bin/sh。

2.2.3. 在 Ubuntu 上使用 Docker

若使用的Ubuntu 版本不满足第 2.1 章的编译环境要求，可以在进行第 3 章的编译和烧录步骤前，执行如下步骤使用Docker 环境。

2.2.3.1. 安装 Docker

如下以 Ubuntu 16.04 主机为例介绍Docker 安装步骤。

1. 执行如下命令更新APT 软件包索引：

```
$sudo apt-get update
```

2. 执行如下命令在 Ubuntu 上安装 Docker：

```
$sudo apt install docker.io
```

3. 执行如下命令验证 Docker 是否安装成功:

```
$sudo docker -v
```

若 Docker 安装成功，执行结果如下图所示:

```
$ sudo docker -v
Docker version 20.10.7, build f0df350
```

2.2.3.2. 将 Ubuntu 18.04/20.04/22.04 镜像导入 Docker

1. 从 <https://developer.quectel.com/doc/files/QSM368ZP/tool.zip> 下载 *smartlinux-docker-ubuntu1804_v2.tar* (Linux 系统)。
2. 执行如下命令将镜像导入 Docker:

- Linux 系统:

```
$ sudo docker load -i smartlinux-docker-ubuntu1804_v2.tar
```

- Ubuntu 系统:

```
$ sudo docker load -i smartlinux-rk3568-ubuntu-20.04-v1.3.tar.gz
```

- OpenWrt 系统:

```
$ sudo docker load -i smartlinux-rk3568-ubuntu-22.04-v1.0.tar.gz
```

3. 执行如下命令查看导入到 Docker 的镜像:

```
$ sudo docker images
```

导入的 Docker 镜像如下图所示:

```
$ sudo docker images
REPOSITORY TAG IMAGE ID CREATED SIZE
rk3568 v2 8b3a763ea933 5 months ago 845MB
```

图 1: 导入到 Docker 的镜像 (Linux 系统)

```
$ sudo docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
rk3568-ubuntu-20.04	v1.3	dc06012462b9	14 minutes ago	1.43GB

图 2: 导入到 Docker 的镜像 (Ubuntu 系统)

```
ivan@Ivan ~ $ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
smartlinux-rk3568-ubuntu-22.04	v1.0	432dcf32c5b2	3 weeks ago	1.32GB

图 3: 导入到 Docker 的镜像 (OpenWrt 系统)

4. 执行如下命令运行镜像 (推荐参考备注 2):

- Linux 系统:

```
$ sudo docker run -it 8b3a763ea933 /bin/bash
```

- Ubuntu 系统:

```
$ sudo docker run --privileged -it dc06012462b9 /bin/bash
```

- OpenWrt 系统:

```
$ sudo docker run --privileged -it 432dcf32c5b2 /bin/bash
```

备注

- 默认已在 Docker 镜像中创建了用户名为“quectel”的账户。
- 若 PC 编译机的根目录下空间不足 (空闲存储 200 GB)，可以将主机中需要的目录映射到 Docker 镜像中。例如，执行如下命令，将主机中的 `/home/quectel` 目录映射到 Docker 镜像。

- Linux 系统:

```
$ cd /home/quectel/
$ mkdir share
$ sudo docker run -it -v /home/quectel/share:/home/test/share 8b3a763ea933 /bin/bash
```

- Ubuntu 系统:

```
$ cd /home/quectel/
$ mkdir share
```

```
$ sudo docker run --privileged -it -v /home/quectel/share:/home/test/share \
dc06012462b9 /bin/bash
```

- OpenWrt 系统:

```
$ cd /home/quectel/
$ cd /home/quectel/
$ mkdir share
$ sudo docker run --privileged -it -v /home/quectel/share:/home/test/share \
432dcf32c5b2 /bin/bash
```

5. 使用 “Ctrl” + “P” 和 “Ctrl” + “Q” 退出 Docker 镜像。
6. 执行如下命令查看 Docker 容器 ID:

```
$ sudo docker ps -a
```

Docker 容器 ID 如下图所示:

```
$ sudo docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
4443cc11518c	8b3a763ea933	"/bin/bash"	About a minute ago	Exited (0) 13 seconds ago		bold_einstein

图 4: Docker 容器 ID (Linux 系统)

```
$ sudo docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
e9d987f699de	dc06012462b9	"/bin/bash"	About a minute ago	Exited (0) 10 seconds ago		pedantic_meitner

图 5: Docker 容器 ID (Ubuntu 系统)

```
ivan@ivan /media/magacin/docker/images $ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
264b70f7c860	432dcf32c5b2	"/bin/bash"	3 minutes ago	Up 2 minutes		reverent_lovelace

图 6: Docker 容器 ID (OpenWrt 系统)

7. 执行如下命令再次进入 Docker 容器:

```
$ sudo docker attach CONTAINER ID
```

8. 执行如下命令停用 Docker 容器:

```
$ sudo docker stop CONTAINER ID
```

9. 若需要重启 Docker 容器，执行如下命令启用 Docker 容器：

```
$ sudo docker start CONTAINER ID
```

备注

1. Docker 镜像上已安装所有工具，用户无需进行任何操作。
2. Docker 镜像默认有两个用户：
 - 用户名：root
 - 用户名：quectel

用户需要添加一个自己的新用户进行编译。例如，通过 **useradd test** 添加名为“test”的用户。登录 Docker bash 时，可以使用 **su test** 切换到“test”用户来构建 SDK。

3. 若在 Ubuntu 系统上编译时报错如下：

```
ERROR: ubuntu-base-20.04-r0 do_ubuntu_basic_configure: Execution of "/tmp-glibc/work/aarch64-oe-linux/ubun
do_ubuntu_basic_configure.792163" failed with exit code 126
ERROR: Logfile of failure stored in: /tmp-glibc/work/aarch64-oe-linux/ubun
4-r0/temp/log.do_ubuntu_basic_configure.792163
Log data follows:
| DEBUG: Executing shell function do_ubuntu_basic_configure
| NOTE: copy fakeroot and fakechroot lib to ubuntu-base
| /usr/sbin/chroot: failed to run command '/bin/bash': Exec format error
| WARNING: /tmp-glibc/work/aarch64-oe-linux/ubun
sic_configure.792163:1 exit 126 from 'fakeroot fakeroot chroot /tmp-glibc/work/aarch
64-oe-linux/ubun
do_ubuntu_basic_configure.792163' failed with exit code 126
ERROR: Execution of '/tmp-glibc/work/aarch64-oe-linux/ubun
do_ubuntu_basic_configure.792163' failed with exit code 126
ERROR: Task ( /tmp-glibc/work/aarch64-oe-linux/ubun
do_ubuntu_basic_configure.792163) failed with exit c
ode '1'
```

此时，在 Docker 之外的主机上，从 <http://archive.ubuntu.com/ubuntu/pool/universe/q/qemu/> 下载最新的“qemu-user-static” deb 包并安装，如下所示：

```
$ wget http://archive.ubuntu.com/ubuntu/pool/universe/q/qemu/
qemu-user-static_6.2+dfsg-2ubuntu6.12_amd64.deb
$ sudo dpkg -i qemu-user-static_6.2+dfsg-2ubuntu6.12_amd64.deb
```

3 编译与烧录

3.1. 访问 GitLab 克隆代码

`https://developer.quectel.com/doc/files/QSM368ZP/rk3568\_linux\_r60\_v1.3.2\_qsm368zp-QSM368ZP\_rl.zip`

3.2. 编译命令

3.2.1. Linux 系统

3.2.1.1. Kernel 5.10 及之前

1. 执行如下命令进行整编：

```
$ cd RK3568_Linux_R60_v1.3.2
$ source build-quec.sh
$ build-all-image
```

编译完成后，生成的镜像位于 *rockdev/*路径下。

2. 执行如下命令清除所有内容：

```
$ cd RK3568_Linux_R60_v1.3.2
$ source build-quec.sh
$ buildclean
```

3. 执行如下命令编译 *uboot*：

```
$ cd RK3568_Linux_R60_v1.3.2
$ source build-quec.sh
$ makeuboot
```

编译完成后，生成的 *uboot.img* 位于 *u-boot/*路径下。

4. 执行如下命令编译 *kernel*：

```
$ cd RK3568_Linux_R60_v1.3.2
$ source build-quec.sh
$ makekernel
```

编译完成后，生成的 *boot.img* 位于 *kernel/*路径下。

5. 执行如下命令编译 *recovery*：

```
$ cd RK3568_Linux_R60_v1.3.2
$ source build-quec.sh
$ makerecovery
```

编译完成后，生成的 *recovery.img* 位于 *buildroot/output/rockchip_rk356x_recovery/images/*路径下。

6. 执行如下命令编译 *rootfs*：

```
$ cd RK3568_Linux_R60_v1.3.2
$ source build-quec.sh
$ makeroofs-buildroot
```

编译完成后，生成的 *rootfs.ext2* 位于 *buildroot/output/rockchip_rk3568/images/*路径下。

3.2.1.2. Kernel 6.1

1. 执行如下命令进行整编:

```
$ cd RK3568_Linux6.1_R62_rkr5
$ source build-quec.sh
$ build-all-image
```

编译完成后, 生成的镜像位于 *rockdev/*路径下。

2. 执行如下命令清除所有内容:

```
$ cd RK3568_Linux6.1_R62_rkr5
$ source build-quec.sh
$ buildclean
```

3. 执行如下命令编译uboot:

```
$ cd RK3568_Linux6.1_R62_rkr5
$ source build-quec.sh
$ makeuboot
```

编译完成后, 生成的 *uboot.img* 位于 *u-boot/*路径下。

4. 执行如下命令编译 kernel:

```
$ cd RK3568_Linux6.1_R62_rkr5
$ source build-quec.sh
$ makekernel
```

编译完成后, 生成的 *boot.img* 位于 *kernel/*路径下。

5. 执行如下命令编译recovery:

```
$ cd RK3568_Linux6.1_R62_rkr5
$ source build-quec.sh
$ makerecovery
```

编译完成后, 生成的 *recovery.img* 位于 *buildroot/output/rockchip_rk356x_recovery/images/*路径下。

6. 执行如下命令编译rootfs:

```
$ cd RK3568_Linux6.1_R62_rkr5
$ source build-quec.sh
$ makeroofs-buildroot
```

编译完成后, 生成的 *rootfs.ext2* 位于 *buildroot/output/rockchip_rk3568/images/*路径下。

3.2.2. Ubuntu 系统

1. 执行如下命令进行整编:

```
$ cd rk3568_ubuntu_r60_v1.3.2
$ source build-quec.sh
$ build-all-image-yocto
```

编译完成后, 生成的镜像位于 *rockdev/*路径下。

2. 执行如下命令清除所有内容:

```
$ cd rk3568_ubuntu_r60_v1.3.2
$ source build-quec.sh
$ buildclean
```

3. 执行如下命令编译uboot:

```
$ cd rk3568_ubuntu_r60_v1.3.2
$ source build-quec.sh
$ makeuboot
```

编译完成后, 生成的 *uboot.img* 位于 *u-boot/*路径下。

4. 执行如下命令编译 kernel:

```
$ cd rk3568_ubuntu_r60_v1.3.2
$ source build-quec.sh
$ makekernel
```

编译完成后, 生成的 *boot.img* 位于 *kernel/*路径下。

5. 执行如下命令编译recovery:

```
$ cd rk3568_ubuntu_r60_v1.3.2
$ source build-quec.sh
```

```
$ makerecovery
```

编译完成后，生成的 *recovery.img* 位于 *buildroot/output/rockchip_rk356x_recovery/images/*路径下。

6. 执行如下命令编译rootfs:

```
$ cd rk3568_ubuntu_r60_v1.3.2
$ source build-quec.sh
$ makerootfs-yocto
```

编译完成后，生成的 *qti-ubuntu-robotics-image-rockchip-rk3568-evb.rootfs.ext4* 位于 *yocto/build/tmp-glibc/deploy/images/rockchip-rk3568-evb/*路径下。

3.2.3. OpenWrt 系统

1. 执行如下命令进行整编:

```
$ cd rk3568_openwrt_r61_v1.4.0
$ source build-quec.sh
$ build-all-image-openwrt
```

编译完成后，生成的镜像位于 *rockdev/*路径下。

2. 执行如下命令清除所有内容:

```
$ cd rk3568_openwrt_r61_v1.4.0
$ source build-quec.sh
$ buildclean
```

3. 执行如下命令编译uboot:

```
$ cd rk3568_openwrt_r61_v1.4.0
$ source build-quec.sh
$ makeuboot
```

编译完成后，生成的 *uboot.img* 位于 *u-boot/*路径下。

4. 执行如下命令编译 kernel:

```
$ cd rk3568_openwrt_r61_v1.4.0
$ source build-quec.sh
$ makekernel
```

编译完成后，生成的 *boot.img* 位于 *kernel/*路径下。

5. 执行如下命令编译recovery:

```
$ cd rk3568_openwrt_r61_v1.4.0
$ source build-quec.sh
$ makerecovery
```

编译完成后，生成的 *recovery.img* 位于 *buildroot/output/rockchip_rk356x_recovery/images/*路径下。

6. 执行如下命令编译rootfs:

```
$ cd rk3568_openwrt_r61_v1.4.0
$ source build-quec.sh
$ makeroofs-buildroot
```

编译完成后，生成的 *rootfs.ext2* 位于 *buildroot/output/rockchip_rk3568/images/*路径下。

备注

1. 由于首次编译SDK时会从网上下载很多软件包，因此编译时间较长。
2. 为节省后续编译用时，可以将下载包保存到计算机上的公共文件夹中，然后更改 *buildroot/Config.in* 中 *BR2_DL_DIR* 的默认路径为公共文件夹路径。若是在Ubuntu系统上编译，还需更改 *yocto/build/conf/include/common.conf* 中 *DL_DIR* 的默认路径为公共文件夹路径；若是在OpenWrt系统上编译，还需更改 *openwrt/config/sg368z_defconfig* 中 *CONFIG_DOWNLOAD_FOLDER* 和 *buildroot/Config.in* 中 *BR2_DL_DIR* 的默认路径为公共文件夹路径。
3. 若从GitLab上更新SDK，建议在编译之前按照上述步骤2清除旧SDK包中的所有内容。

3.3. 构建、加载及烧录固件包

1. 从 <https://developer.quectel.com/doc/files/QSM368ZP/tool.zip> 下载 *RKDevTool_Release_v2.92.zip* 软件包。
2. 将Linux、Ubuntu或OpenWrt镜像文件（*rockdev/*目录下编译生成的镜像）复制到 *RKDevTool_Release/*目录下，包含的文件如下图所示：

ool > RKDevTool_Release > rockdev				搜索"rockdev"
名称	修改日期	类型	大小	
boot.img	2024/6/29 13:25	IMG 文件	41,775 KB	
linux-headers.tar	2024/6/29 13:26	WinRAR 压缩文件	80,190 KB	
MiniLoaderAll.bin	2024/6/29 13:19	QXDM BIN file t...	435 KB	
misc.img	2024/6/29 13:18	IMG 文件	48 KB	
nvdata1.img	2024/6/29 13:18	IMG 文件	48 KB	
nvdata2.img	2024/6/29 13:18	IMG 文件	48 KB	
oem.img	2024/6/29 13:25	IMG 文件	17,048 KB	
parameter.txt	2024/6/24 19:50	TXT 文件	1 KB	
recovery.img	2024/6/29 13:25	IMG 文件	48,442 KB	
rootfs.img	2024/6/29 13:24	IMG 文件	791,552 KB	
uboot.img	2024/6/29 13:19	IMG 文件	4,096 KB	
update.img	2024/6/29 13:25	IMG 文件	908,207 KB	
userdata.img	2024/6/29 13:25	IMG 文件	4,368 KB	

图 7：镜像文件

3. 进入“**LOADER**”下载模式：将 EVB 开发板上的“**BOOT**”开关拨动至“**ON**”，如下图所示。然后，启动 EVB 并给设备上电，设备将进入 USB Loader 状态。

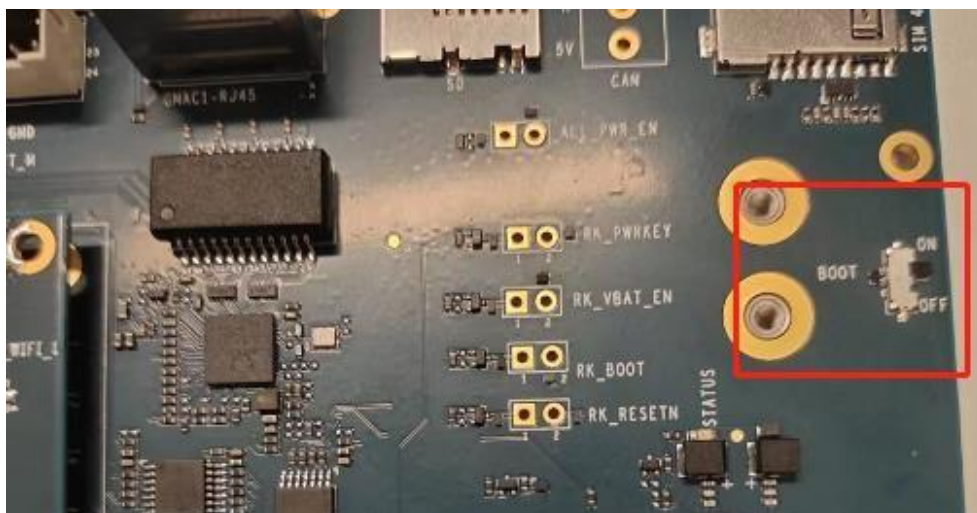


图 8：BOOT 开关

4. 运行RKDevTool 工具，在工具中的分区表空白处单击右键，在弹出的对话框中选择“导入配置”，选择 `RKDevTool_Release/rockdev/parameter.txt` 文件，选择后将自动加载设备的分区配置信息，左键分别单击各分区表格最右侧按键（下图中的红框处），根据分区名称选择对应镜像文件。



图 9：固件选择

备注

- 1. *persist* 分区保存Wi-Fi/BT 相关射频参数，禁止选择编译生成的 *persist.img* 进行烧录。
- 2. *nvdata1* 和 *nvdata2* 分区保存出厂参数，禁止选择编译生成的 *nvdata1.img* 和 *nvdata2.img* 进行烧录。

5. 点击“执行”按钮，等待固件包烧录完成：

```
测试设备开始
测试设备成功
正在下载Gpt... (100%)
开始下载uboot...
正在下载 uboot... (100%)
开始下载misc...
正在下载 misc... (100%)
开始下载boot...
正在下载 boot... (100%)
开始下载recovery...
正在下载 recovery... (100%)
开始下载rootfs...
正在下载 rootfs... (100%)
开始下载oem...
正在下载 oem... (100%)
开始下载userdata...
正在下载 userdata... (100%)
下载完成
```

图 10：烧录结果

6. 固件包烧录完成后，设备将自动重启。为了能正常进入设备系统，设备重启前，需要手动将 EVB 开发板上的“**BOOT**”开关拨动至“**OFF**”。

备注

设备的分区表文件路径为 `device/rockchip/rk356x/parameter-buildroot-fit.txt`。请勿自行修改默认分区，如需修改，请联系移远通信技术支持。

4 开发示例

4.1. Linux 系统下的开发示例

进入代码顶层目录，在shell中执行如下 **source** 命令：

```
$ source build-quec.sh
```

执行如下命令单独编译.mk文件：

```
$ envsetup_buildroot
```

1. 通过 CMakeLists 编译 helloworld

- 源码： *external/quectel-app/helloworld/*
- .mk 文件配置： *buildroot/package/quectel-app/helloworld/helloworld.mk*
- Config.in 文件配置： *buildroot/package/quectel-app/helloworld/Config.in*

执行如下命令编译 *helloworld*：

```
$ remake helloworld
```

编译完成后，生成的文件位于 *buildroot/output/rockchip_rk3568/build/helloworld-1.0.0/*目录下。

2. 通过 Autoconf 编译 apptest

- 源码： *external/quectel-core/apptest*
- .mk 文件配置： *buildroot/package/quectel-core/apptest/apptest.mk*
- Config.in 文件配置： *buildroot/package/quectel-core/apptest/Config.in*

执行如下命令编译 *apptest*:

```
$ remake apptest
```

编译完成后，生成的文件位于 *buildroot/output/rockchip_rk3568/build/apptest-1.0.0/*目录下。

3. 默认安装 *helloworld* 和 *apptest* 到系统

在 *buildroot/configs/quectel/quectel_core.config* 和 *buildroot/configs/quectel/quectel_app.config* 文件中打开编译宏，如下图所示：

```
1 BR2_PACKAGE_QUECTEL_CORE=y
2 BR2_PACKAGE_APPTTEST=y
```

```
1 BR2_PACKAGE_QUECTEL_APP=y
2 BR2_PACKAGE_HELLOWORLD=y
```

图 11: 打开编译宏

此时，执行如下命令编译所有内容：

```
$ build-all-image
```

备注

有关Buildroot的更多信息，请访问链接 <https://buildroot.org/>。

4.2. Ubuntu 系统下的开发示例

进入代码顶层目录，在shell中执行如下 **source** 命令：

```
$ source build-quec.sh
```

执行如下命令单独编译.bb文件：

```
$ envsetup_yocto
```

1. 通过 CMakeLists 编译 helloworld

- 源码: `external/quectel-app/example/helloworld/`
- .bb 文件配置: `yocto/meta-quectel-app/recipes/example/helloworld_0.1.bb`

执行如下命令编译 `helloworld`:

```
$ rebake helloworld
```

编译完成后, 生成的文件位于 `build/tmp-glibc/work/aarch64-oe-linux/helloworld/0.1-r0/`目录下。

2. 通过 Autoconf 编译 apptest

- 源码: `external/quectel-app/example/apptest/`
- .bb 文件配置: `yocto/meta-quectel-app/recipes/example/apptest_0.1.bb`

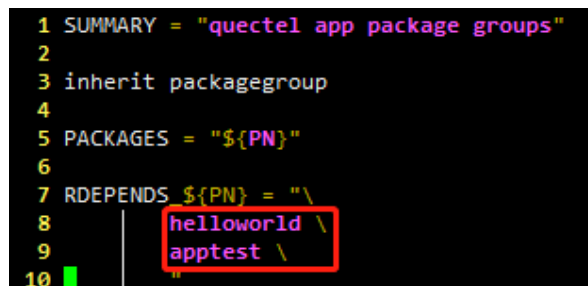
执行如下命令编译 `apptest`:

```
$ rebake apptest
```

编译完成后, 生成的文件位于 `build/tmp-glibc/work/rockchip_rk3568_evb-oe-linux/apptest/0.1-r0/`目录下。

3. 默认安装 helloworld 和 apptest 到系统

取消 `yocto/meta-quectel-app/recipes/packagegroups/packagegroup-quectel-app.bb` 红框内代码的注释, 将 `helloworld` 和 `apptest` 安装到系统, 如下图所示:



```
1 SUMMARY = "quectel app package groups"
2
3 inherit packagegroup
4
5 PACKAGES = "${PN}"
6
7 RDEPENDS_${PN} = "\
8 | helloworld \
9 | apptest \
10 |"
```

图 12: 示例

此时, 执行如下命令编译所有内容:

```
$ build-all-image-yocto
```

备注

有关 Yocto 详细信息，请访问链接 <https://www.yoctoproject.org/>。

4.3. OpenWrt 系统下的开发示例

进入代码顶层目录，在shell 中执行如下 **source** 命令：

```
$ source build-quec.sh
```

1. 通过 Makefile 编译 helloworld

- 源码：external/quectel-app/helloworld/src/
- Makefile 文件配置：openwrt/package/quectel-app/helloworld/Makefile

进入 openwrt/目录，执行如下命令进入配置界面：

```
make menuconfig
```

配置界面如下，找到 *Helloworld* 包并按 “**Enter**” 键进行选择，如下图所示：

```

Target System (Rockchip) --->
Subtarget (RK33xx boards (64 bit)) --->
Target Profile (FriendlyARM NanoPi R2S) --->
Target Images --->
[ ] Enable experimental features by default
Global build settings --->
[*] Advanced configuration options (for developers) --->
[ ] Build the OpenWrt Image Builder
[ ] Build the OpenWrt SDK
[ ] Package the OpenWrt-based Toolchain
[ ] Image configuration --->
Base system --->
Administration --->
Boot Loaders --->
Development --->
Extra packages --->
Firmware --->
Fonts --->
Helloworld --->
Kernel modules --->
Languages --->
Libraries --->
LuCI --->
Mail --->
Multimedia --->
Network --->
quectel --->
Quectel app package --->
Quectel core package --->
Rockchip --->
v(+)

<Select> <Exit> <Help> <Save> <Load>

```

```
<*) helloworld..... Helloworld -- by Quectel
```

选择 *Helloworld* 包后，*openwrt/config* 文件将会更新，保存修改后的配置并返回代码顶层目录，如下图所示：

```
2832 #
2833 # Helloworld
2834 #
2835 CONFIG_PACKAGE_helloworld=y
2836 # end of Helloworld
```

将修改内容添加到 *sg368z_defconfig* 文件中，如下图所示：

```
6992 #
6993 # Helloworld
6994 #
6995 CONFIG_PACKAGE_helloworld=y
6996 # end of Helloworld
6997
```

此时，执行如下命令编译所有内容：

```
$ build-all-image-openwrt
```

编译完成后，检查二进制文件是否已成功编译，如下图所示：

```
rk@Ivan:/code/RK3568_R61/RK3568 Linux R61 v1.4.0$ ls openwrt/build_dir/target-aarch64_generic_glibc/root-rockchip/bin/
ash          chgrp        cp            dmesg         fgrep         azis          ln            mkdir         mv            opkg          ping6         rm            sh            tar            true          uname
board_detect chmod         date          echo           fsync         helloworld   lock          mknod         netmsg        passwd        ps            rmdir         sleep         touch         ubus          vi
busybox      chown        dd            egrep          grep          ipcalc.sh    login         mktemp        netstat       pidof         pwd           sed           stty          traceroute    uclient-fetch zcat
cat          config_generate df             false          gunzip        kill          ls            mount         nice          ping          quectel-CM    sendat        sync          traceroute6   umount
```

备注

有关Buildroot 详细信息，请访问链接 <https://buildroot.org/>。

5 附录 参考文档及术语缩写

表 2: 参考文档

文档名称
[1] Quectel_智能模块_GitLab_用户指导

表 3: 术语缩写

缩写	英文全称	中文全称
BT	Bluetooth	蓝牙
CPU	Central Processing Unit	中央处理器
EVB	Evaluation Board	评估板
FTP	File Transfer Protocol	文件传输协议
ID	Identifier	标识符
LTS	Transport Layer Security	传输层安全（协议）
OS	Operating System	操作系统
PC	Personal Computer	个人计算机
RF	Radio Frequency	射频
SDK	Software Development Kit	软件开发工具包
SSD	Solid State Disk/Drive	固态硬盘/驱动器
SSH	Secure Shell	安全外壳协议
USB	Universal Serial Bus	通用串行总线